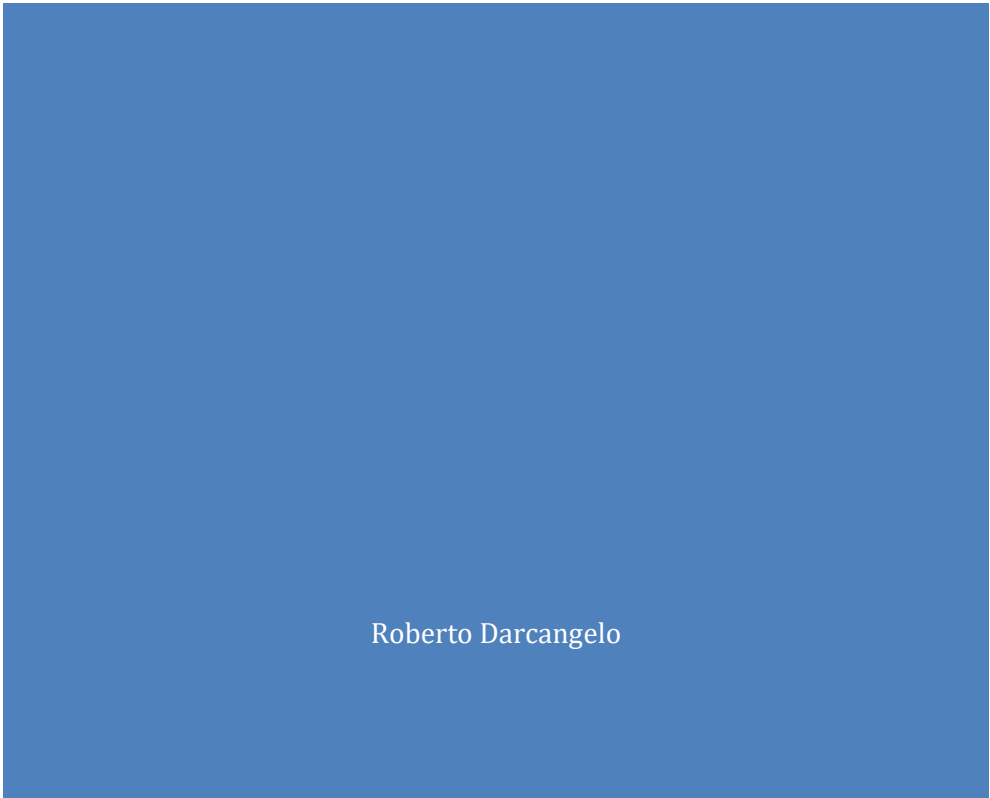




HOW MACHINES LEARN MADE SIMPLE



Roberto Darcangelo

Indice

Indice.....1

Prefazione.....3

Prerequisiti4

Fondamenti del Machine Learning5

 1.1 Introduzione.....5

 1.2 Cosa usa la macchina6

 1.3 Cos'è un modello di machine learning.....7

 1.4 Tipi di apprendimento10

 1.4.1 Apprendimento Supervisionato10

 Problemi di classificazione10

 Problemi di regressione.....11

 1.4.2 Apprendimento non supervisionato12

 Clustering.....12

 Il clustering esclusivo.....13

 Il clustering sovrapposto.....14

 Il clustering gerarchico.....14

 1.4.3 Riduzione della dimensionalità19

 PCA (Analisi dei componenti principali).....21

 1.4.4 Apprendimento di Rinforzo22

 1.4.5 Apprendimento Semi-Supervisionato25

1.5 Valutare un modello di Machine Learning.....	27
Dividere i dati	27
1.5.1 Valutazione dei modelli di classificazione.....	28
1.5.2 Valutazione dei modelli di regressione	31
1.6 Come impara una macchina.....	32
1.6.1 Funzioni di perdita.....	32
1.6.2 Aggiornamento dei parametri.....	35
Capitolo 2 Modelli di Machine Learning Supervisionato	36
2.1 Regressione Lineare.....	36
Progetto 1 Regressione Lineare da zero con NumPy.....	38
Progetto 2 Regressione Lineare con Sklearn e Pandas	43
2.2 Regressione Logistica.....	50
Regressione Logistica From Scratch con NumPy	52
Regressione Logistica con Sklearn e Pandas	55

Prefazione

Sin da piccolo, sono sempre stato amante dei computer, grazie a mio padre, che mi ha insegnato tutte le nozioni che ancora oggi uso, a 12 anni iniziai a studiare Python, il linguaggio ad alto livello più diffuso al mondo.

Ma nel 2025 all'età di 16 anni, nel bel pieno dell'ondata dell'IA ho iniziato a sviluppare sul serio. Ho iniziato sviluppando siti web con HTML, CSS e JS per poi passare allo sviluppo di applicazioni mobile. Tutto cambiò quando il mio professore a me ed un mio amico dopo la lezione iniziò a parlare di Reti Neurali, Machine Learning, da lì mi innamorai di quel mondo che mi sembrava surreale, insomma io scrivevo semplicemente applicazioni mobile e vedere che lo stesso computer con cui scrivevo le applicazioni potesse fare quei calcoli, mi stupiva. Da quel momento ho iniziato a studiare in modo continuo, leggendo libri, paper accademici, ma senza un percorso preciso. Col tempo poi ho iniziato a darmi un metodo ogni giorno scrivevo codice e imparavo nozioni, consumavo quaderni per scrivere algoritmi e nozioni di matematica anche extra-scolastiche, quindi ho deciso di scrivere questo libro per trasmettere ciò che imparo nel modo più semplice possibile.

Prerequisiti

Per approcciare questo libro bisogna avere una conoscenza base sul linguaggio Python in modo da avere facilità nel comprendere i progetti presenti nel libro. Non è richiesta una competenza nella scienza dei dati perché andremo a ricoprire quel campo durante i progetti con spiegazioni dettagliate.

Fondamenti del Machine Learning

1.1 Introduzione

L'intelligenza artificiale si occupa di sviluppare sistemi in grado di apprendere relazioni dai dati per prendere decisioni o fare previsioni. Il machine learning è un sottoinsieme dell'intelligenza artificiale che utilizza algoritmi capaci di apprendere automaticamente dai dati senza essere programmati esplicitamente. In questo libro partiremo dal machine learning classico, per poi arrivare al deep learning, che rappresenta un'evoluzione di questi metodi.

Nel metodo tradizionale di classificazione di immagini, un programmatore definisce manualmente le caratteristiche (feature) come occhi, orecchie o forma del volto.

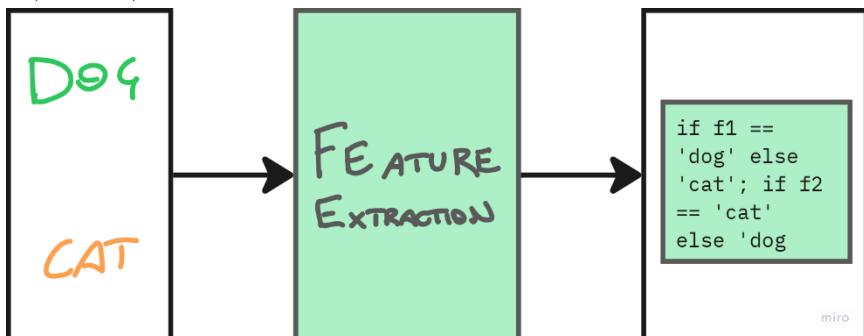


Figura 1.1 Mostra il metodo tradizionale che senza machine learning classifica se l'immagine rappresenta un gatto o un

cane, dove $f_1...f_n$ sono le caratteristiche dell'animale rappresentato, come: occhi, orecchie, baffi.

Con il machine learning, il modello utilizza queste caratteristiche per apprendere la relazione tra input e output. Il deep learning rappresenta un passo ulteriore: le reti neurali apprendono direttamente dai dati grezzi, senza bisogno di definire manualmente le caratteristiche. Il primo passo per capire questo processo è il machine learning classico, che introduce i concetti fondamentali su cui si basa il deep learning.

1.2 Cosa usa la macchina

Per capire come funziona il machine learning, bisogna prima capire con cosa lavorano le macchine. Le macchine lavorano con i dati. Un dato è una singola informazione elementare, come un numero, una parola o un valore, che da solo non ha ancora un significato utile finché non viene interpretato insieme ad altri dati. In altre parole, il dato è la forma più semplice di informazione grezza, cioè non ancora organizzata o collegata a uno scopo preciso. Quando più dati vengono raccolti, organizzati e strutturati, diventano informazione. Un insieme ordinato di dati prende il nome di dataset. Un dataset può essere immaginato come una tabella, simile a un foglio Excel. Questa tabella è composta da righe e colonne. Ogni riga

rappresenta un esempio, cioè un singolo caso del problema che vogliamo studiare. Ogni colonna invece rappresenta una caratteristica di quell'esempio, chiamata feature. Le feature sono le informazioni che descrivono un elemento del dataset. Per capirlo meglio, immaginiamo un dataset sugli animali domestici. Ogni animale è una riga della tabella, mentre le sue caratteristiche sono le colonne. Ad esempio, possiamo avere feature come il numero di zampe, il colore del pelo, la presenza della coda o la taglia dell'animale. Insieme, queste informazioni descrivono completamente ogni esempio. Oltre alle feature, in molti casi troviamo anche una colonna speciale chiamata label o target. La label è la risposta corretta associata a ogni esempio. È ciò che vogliamo che il modello impari a prevedere. Nel caso degli animali, la label potrebbe essere “gatto” oppure “cane”. Questa informazione è fondamentale perché serve al modello per capire quale relazione esiste tra le caratteristiche e la risposta finale. Quando la label è presente, il dataset è detto “etichettato” e verrà utilizzato nell'apprendimento supervisionato, dove il modello impara proprio confrontando le sue previsioni con le risposte corrette.

1.3 Cos'è un modello di machine learning

Un modello di machine learning è un sistema matematico che impara a trasformare dei dati in una risposta. Riceve delle

informazioni in ingresso e produce un risultato in uscita cercando di capire la relazione tra le due cose. Per immaginarlo in modo semplice possiamo pensarlo come una scatola che riceve dati e restituisce una previsione. All'inizio questa scatola non ha conoscenza. Se gli diamo un input la sua risposta è casuale o completamente sbagliata. Dentro questa scatola non c'è magia ma ci sono dei parametri interni. I parametri sono numeri che controllano il comportamento del modello e determinano come vengono trasformati gli input in output. Il compito del modello è imparare i valori giusti di questi parametri. Il processo di apprendimento funziona in modo iterativo. Il modello fa una previsione, poi confronta il risultato con la risposta corretta presente nel dataset. Da questo confronto si calcola un errore chiamato loss che indica quanto la previsione è lontana dal valore giusto. Se l'errore è grande significa che il modello sta sbagliando molto, se è piccolo significa che sta prevedendo correttamente. A questo punto un algoritmo interviene per modificare leggermente i parametri interni del modello in modo da ridurre l'errore. In questo modo il modello si aggiusta gradualmente ogni volta che sbaglia. Questo processo viene ripetuto molte volte su tanti esempi del dataset. Il modello continua a fare previsioni, sbagliare e correggersi fino a quando i suoi parametri diventano abbastanza buoni da rappresentare le relazioni tra i dati. Alla fine dell'addestramento il modello non è più casuale. Ha imparato una configurazione di parametri che gli permette di

fare previsioni corrette anche su dati che non ha mai visto prima.

1.4 Tipi di apprendimento

Ora che sappiamo cosa sono i dataset e cos'è un modello possiamo capire come la macchina impara con i diversi tipi di apprendimento, questi sono scelti a seconda dei dati che abbiamo e all'obiettivo del problema. Esistono diversi tipi di apprendimento nel machine learning e sono chiamati: Apprendimento supervisionato, apprendimento non supervisionato, apprendimento di rinforzo e apprendimento semi-supervisionato.

1.4.1 Apprendimento Supervisionato

L'apprendimento supervisionato prevede l'allenamento di un modello di machine learning su dati etichettati, ovvero dati che presentano la **label** quindi la risposta corretta che il modello deve prevedere. Questo tipo di apprendimento si usa principalmente per due problemi

Problemi di classificazione

Sono problemi in cui il modello deve assegnare un dato a una categoria.

Esempi di classificazione sono: classificare se un email è spam o no, se una transazione bancaria è una frode o no. Questi esempi fanno parte della **classificazione binaria** (Sì/No), dove dobbiamo categorizzare il dato in solo due classi.

Mentre ad esempio se vogliamo classificare se in un'immagine è presente un gatto, un uccello o un cane dobbiamo usare modelli adatti alla **classificazione multinomiale**, che è un tipo di classificazione dove dobbiamo categorizzare il dato in 3 o più classi.

Problemi di regressione

I problemi di regressione trattano la previsione di valori numerici, continui in base ai dati di input, ad esempio se volessimo prevedere il prezzo di una casa in base ai metri quadri della casa.

L'apprendimento supervisionato è molto diffuso nel mondo reale, i problemi di classificazione vengono usati nelle aziende per prevedere se un cliente abbandonerà il servizio dell'azienda o meno, tutto in base al suo storico.

La regressione altrettanto, usata dalle compagnie aeree per prevedere il prezzo del biglietto più conveniente alla compagnia in base a destinazione e dati storici.

1.4.2 Apprendimento non supervisionato

L'apprendimento non supervisionato al contrario di quello supervisionato prevede l'allenamento di un modello di machine learning su dati non etichettati. Questi modelli scoprono pattern nascosti o raggruppamenti di dati senza l'intervento umano. Viene usato ampiamente nella cosiddetta nella segmentazione dei clienti, raggruppando clienti in base al loro storico di acquisti, attività sui social media o posizione geografica.

Gli approcci più comuni dell'apprendimento non supervisionato sono:

Clustering

Questa sezione è più dettagliata perché il clustering è uno degli strumenti principali dell'apprendimento non supervisionato. Il **clustering** è un approccio che mira a raggruppare dati in base alle loro somiglianze, possiamo pensare come esempio, se volessimo organizzare il guardaroba tenderemmo a mettere i maglioni da una parte, i pantaloni da un'altra e così via. Il clustering aiuta anche ad identificare caratteristiche rilevanti che possono essere utilizzate per allenare modelli di machine learning. Abbiamo diversi tipi di clustering ma i più semplici e basilari si dividono in: esclusivi, sovrapposti o gerarchici.

Il clustering esclusivo

E' una forma di raggruppamento che raggruppa un dato in un singolo cluster, per usare questo metodo bisogna specificare quanti K gruppi creare. Per determinare se un dato appartiene a un determinato cluster bisogna calcolare la distanza del dato dal suo centroide più vicino. Un valore di K più alto significherà che ci saranno più gruppi al contrario avremo meno gruppi.

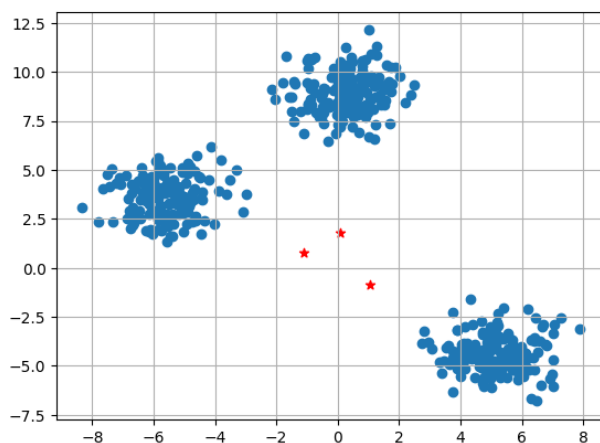


Figura 1.2 In questa figura è rappresentato un raggruppamento di dati, con i rispettivi centroidi (categorie) rappresentati dalle stelle rosse.

Il clustering sovrapposto

In questo tipo di clustering invece i dati possono appartenere anche a più gruppi ma con diversi gradi di appartenenza ad ogni gruppo, algoritmi come il fuzzy K-mean sono usati. Questo significa che un dato non appartiene esclusivamente a un cluster, ma ha una probabilità o un "peso" di appartenenza a ciascuno. Ad esempio, un cliente potrebbe essere al 70% nel cluster 'young spender' e al 30% nel cluster 'price-conscious buyer'. Fuzzy K-means assegna questi gradi automaticamente.

Il clustering gerarchico

Funziona in modo agglomerativo o divisivo, rispettivamente, nel primo approccio si parte considerando ogni elemento come un cluster separato e iterando, si uniscono cluster più simili fino a formare un unico cluster. Nel secondo approccio si parte in un grande cluster che viene progressivamente suddiviso fino a quando tutti i cluster sono singoli. Questi algoritmi di clustering possono decidere autonomamente quando smettere di creare cluster.

Per calcolare la distanza questi algoritmi utilizzano la matrice di dissomiglianza per decidere quali cluster bisogna dividere

o unire. La dissomiglianza è la distanza tra due punti dati, misurata tramite metodi di collegamento (linkage).

Collegamento singolo

Dati due insiemi A e B questo metodo di linkage analizza le distanze a coppie tra gli elementi in due cluster utilizzando la loro distanza minima. Questo metodo è sensibile al rumore e soggetto al concatenamento, ovvero un effetto dove alcuni punti danno vita alla fusione di due cluster in uno.

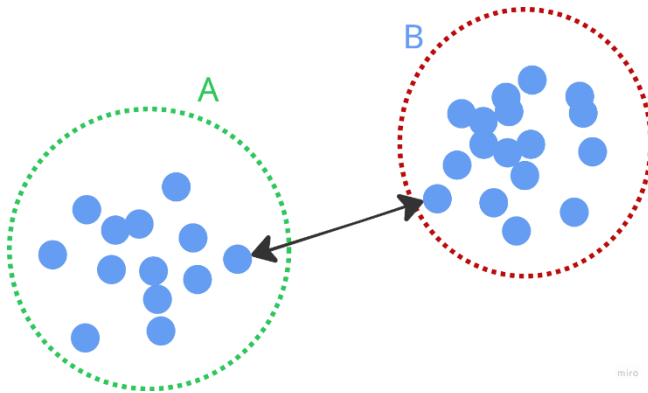


Figura 1.3 In questa figura è rappresentato il metodo di collegamento singolo in cui i punti colorati in blu sono i punti dati e la freccia rappresenta la distanza minima tra due punti di cluster diversi.

Collegamento completo

Dati i due cluster A e B, il metodo di collegamento completo restituisce la **distanza massima** tra i due punti. Questo metodo è meno sensibile al rumore rispetto al metodo di collegamento singolo, ma il suo utilizzo può distorcere i risultati quando ci sono cluster globulari o di **grandi dimensioni**.

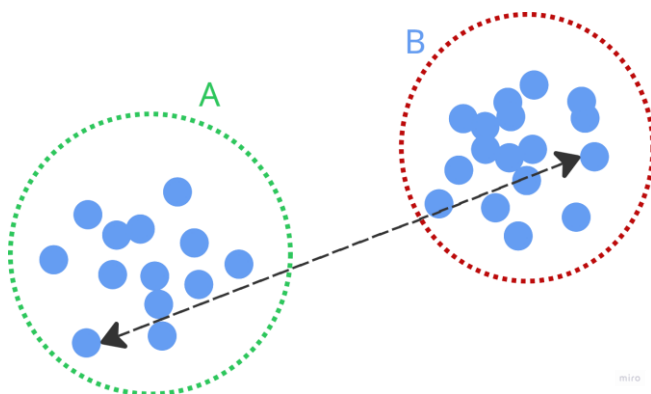


Figura 1.4 In questa figura è rappresentato il metodo di collegamento completo in cui i punti colorati in blu sono i punti dati e la freccia rappresenta la distanza massima tra due punti di cluster diversi.

Collegamento medio

Nel metodo medio, la distanza tra due cluster non si calcola prendendo il punto più vicino (come nel singolo) o il più lontano (come nel completo), ma facendo una **media di tutte le distanze tra i punti dei due cluster**. Immagina due gruppi di persone. Invece di guardare solo la coppia più vicina o più lontana, prendi **tutte le possibili coppie tra i due gruppi** e calcoli la distanza media. Se questa distanza media è bassa, i cluster sono simili quindi vengono uniti. Se è alta restano separati.

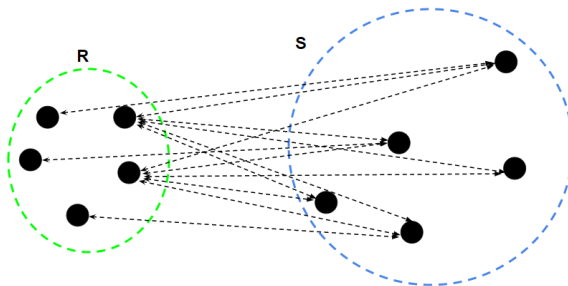


Figura 1.5 In questa figura è rappresentato il metodo di collegamento medio in cui i punti colorati in nero sono i punti dati e le frecce rappresenta la distanza tra le coppie di punti.

Prima di applicare questi metodi, è spesso utile semplificare i dati riducendo il numero di dimensioni.

1.4.3 Riduzione della dimensionalità

Per capire il concetto della riduzione della dimensionalità bisogna prima capire cosa significa dimensione. Immagina ogni dato come un punto in uno spazio. Ogni caratteristica che descrive quel dato rappresenta una direzione in quello spazio, cioè una dimensione.

Per esempio, se descrivi una casa usando solo metri quadri e numero di stanze, stai lavorando in uno spazio a due dimensioni. Ogni casa può essere rappresentata come un punto su un piano. Se aggiungi altre caratteristiche come prezzo, distanza dal centro o anno di costruzione, lo spazio diventa sempre più grande e complesso. Ogni nuova informazione aggiunge una nuova dimensione.

Il problema è che noi possiamo visualizzare facilmente solo due o tre dimensioni, mentre i modelli di machine learning lavorano con decine o migliaia di dimensioni. Questo rende i dati difficili da interpretare e spesso introduce informazioni ripetute o poco utili.

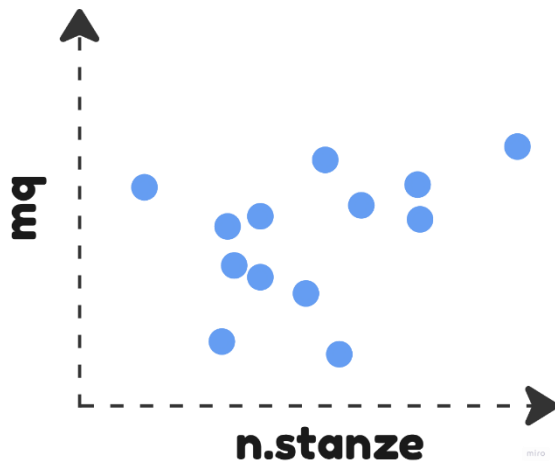


Figura 1.6 Punti dati in uno spazio a due dimensioni.

Molti pensano che avere più caratteristiche, migliori sempre le prestazioni del modello, ma non è sempre vero. Quando le feature diventano troppe, il modello può imparare troppo bene i dati di allenamento e iniziare a memorizzarli invece di generalizzare. Questo fenomeno si chiama **overfitting**. In pratica il modello diventa molto preciso sui dati già visti ma sbaglia su nuovi dati perché ha imparato dettagli troppo specifici e non il pattern generale.

Per questo motivo si usa la riduzione della dimensionalità, che serve a ridurre il numero di feature mantenendo però le informazioni più importanti. L'obiettivo è semplificare i dati senza perdere il significato principale.

PCA (Analisi dei componenti principali)

La PCA è un algoritmo che trasforma le feature originali in nuove feature chiamate componenti principali. Queste nuove componenti non sono semplici copie delle vecchie colonne ma combinazioni delle informazioni originali. Per capire l'idea, immagina che i dati si distribuiscano in una nuvola di punti. La PCA cerca la direzione in cui questi punti sono più sparsi. Questa direzione viene chiamata prima componente principale e contiene la maggior quantità di informazione possibile. In termini semplici significa che descrive il comportamento principale dei dati. La seconda componente principale rappresenta una direzione diversa dalla prima. Non ripete le stesse informazioni ma descrive un altro aspetto dei dati che non era ancora stato catturato. In questo modo ogni componente aggiunge informazione nuova senza ridondanza.

Alla fine del processo, invece di usare tutte le feature originali, si possono usare solo le prime componenti principali come nuove variabili. Per esempio, se si parte da tre feature, si possono ottenere due componenti principali che riassumono quasi tutta l'informazione utile. Questo rende i dati più semplici e spesso migliora anche le prestazioni dei modelli di machine learning.

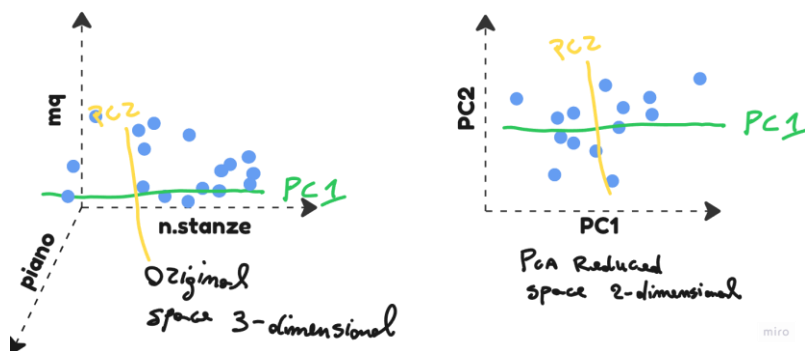


Figura 1.7 Riduzione delle feature grazie al PCA, lo schema a sinistra presenta 3 dimensioni numero di stanze, piano e mq, mentre quella a destra 2 dimensioni, ovvero PC1 e PC2 che PCA ha calcolato.

1.4.4 Apprendimento di Rinforzo

L'apprendimento per rinforzo è un tipo di processo in cui agenti autonomi imparano a prendere decisioni interagendo con il loro ambiente indipendentemente dall'istruzione diretta da parte di un utente umano, ne fanno parte robot e le auto con guida autonoma. A differenza dei metodi di apprendimento supervisionato e non supervisionato, l'apprendimento per rinforzo impara ad agire. Suppone che i dati siano una sequenza ordinata di dati, organizzati come stato-azione-ricompensa. Molti algoritmi di apprendimento per rinforzo imitano l'apprendimento biologico. Un esempio

concreto è quello dell'addestramento del cane, immaginiamo di dover addestrare un cane per la prima volta e vogliamo che il cane si sieda a terra. Il cane riceverà un premio quando eseguirà il comando di sedersi, ma verrà "punito" ogni qual volta non si sieda. Questo processo di apprendimento è esattamente identico al nostro, ovvero agendo attraverso le interazioni con l'ambiente per ottenere il risultato sperato. Abbiamo delle componenti fondamentali dell'apprendimento di rinforzo:

1. Ambiente: Spazio dove opera l'agente
2. Stato dell'agente
3. Ricompensa: Feedback positivo
4. Strategia: Definisce le azioni dell'agente
5. Valore: Ricompensa che riceverà l'agente compiendo un'azione

L'ambiente rappresenta il contesto fisico o virtuale in cui l'agente opera, mentre lo stato descrive la condizione in cui l'agente si trova. Ogni azione influisce sull'ambiente e viene valutata attraverso un sistema di ricompense.

La strategia definisce le azioni da intraprendere in base allo stato attuale, mentre il valore rappresenta una stima delle ricompense future ottenibili eseguendo determinate azioni. Ci sono due metodi con cui un agente raccoglie i dati per l'apprendimento:

1. Online Learning: In questo caso, un agente raccoglie i dati interagendo direttamente con l'ambiente e vengono elaborati in modo iterativo man mano che

l'agente interagisce con l'ambiente. Questi modelli vengono allenati sequenzialmente, in piccoli gruppi di dati chiamati mini-batches, quindi ogni step di allenamento è veloce ed economico, il sistema impara sui nuovi dati appena arrivano.

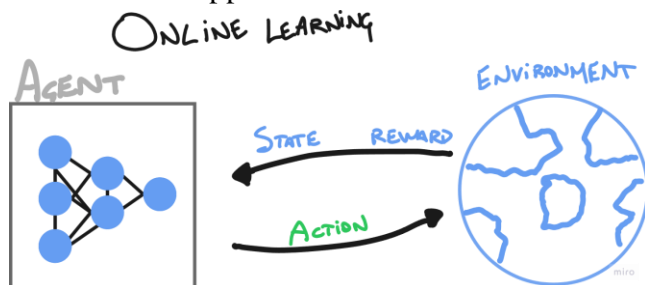
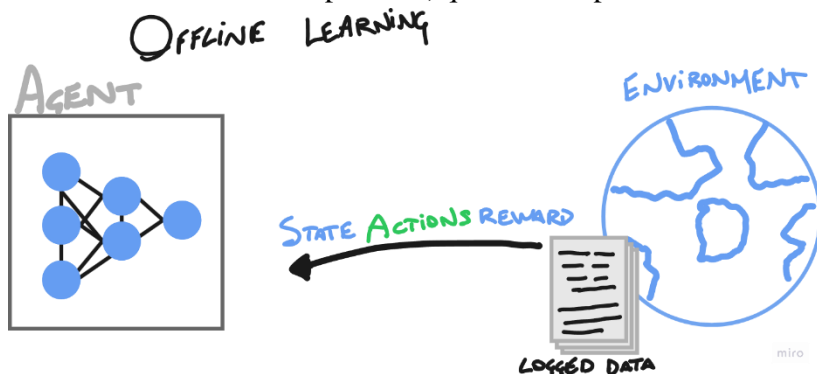


Figura 1.8 Nello schema Online Learning, vedi che l'agente interagisce continuamente con l'ambiente e impara in tempo reale.

2. Offline Learning: Quando un agente non ha accesso diretto ad un ambiente ma solo ai dati registrati. Questo metodo è chiamato così perché il sistema non è capace di imparare continuamente, infatti dev'essere allenato su tutti i dati disponibili, questo comporta un



costo computazionale alto e tempi di allenamento alti. Infatti se noi riceviamo un nuovo tipo di dato nel dataset dovremo riallenare il modello su tutti i dati e non solo sui nuovi.

Figura 1.9 Nello schema Offline Learning, invece, i dati sono registrati e il modello li processa tutti insieme.

1.4.5 Apprendimento Semi-Supervisionato

Questo tipo di apprendimento combina apprendimento supervisionato e apprendimento non supervisionato. In questa tipologia di apprendimento vengono utilizzati sia dati etichettati che non etichettati. Questi tipi di modelli possono combinare sia tecniche di apprendimento non supervisionato come il clustering che abbiamo visto che tecniche di apprendimento supervisionato. Un esempio molto comune è il campo medico.

Nel contesto della diagnosi medica, ad esempio nell'analisi di radiografie, i medici spesso hanno a disposizione un grande numero di immagini ma solo una piccola parte di esse è etichettata con la diagnosi corretta. In questo scenario si utilizza una tecnica chiamata active learning. Il modello viene inizialmente addestrato sui pochi dati etichettati disponibili e poi analizza le immagini non etichettate. In base a quanto è incerto sulle sue previsioni, il modello suggerisce ai radiologi quali immagini dovrebbero essere analizzate per prime. In questo modo si ottimizza il lavoro umano, concentrandosi sui

casi più difficili, e si migliora progressivamente il modello aggiungendo nuove etichette ai dati più utili.

1.5 Valutare un modello di Machine Learning

Dividere i dati

Per capire se un modello di machine learning sta funzionando bene non basta addestrarlo sui dati, ma capire quanto il modello sia “bravo a generalizzare”, cioè fare previsione corrette anche sui dati che non ha visto prima. Per questo motivo i dati non vengono mai usati tutti insieme per l'allenamento. Una parte viene usata per addestrare il modello, mentre un'altra parte viene tenuta sperata per testarlo e valutarlo.

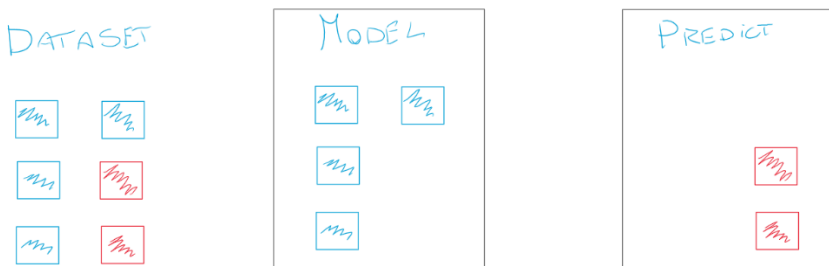


Figura 1.10 Nello schema viene rappresentata la tecnica di train/test split ovvero la tecnica che si utilizza per suddividere i dati da passare al modello.

1.5.1 Valutazione dei modelli di classificazione

Nei problemi di classificazione vogliamo capire quanto il modello è bravo a distinguere tra classi diverse, come “passato/non passato” o “spam/non spam”.

Un primo metodo è l'accuratezza, cioè la percentuale di previsioni corrette rispetto al totale. Tuttavia, da sola non è sempre sufficiente.

Per avere una visione più completa si utilizza la **matrice di confusione**, che confronta le previsioni del modello con i valori reali:

TRUE LABEL y	PASS	5	2
	FAIL	1	4
	PREDICTED VALUES x	PASS	FAIL

Figura 1.11 Matrice di confusione.

True Positive: il modello predice positivo e il risultato è positivo

True Negative: il modello predice negativo e il risultato è negativo

False Positive: il modello predice positivo ma è negativo

False Negative: il modello predice negativo ma è positivo

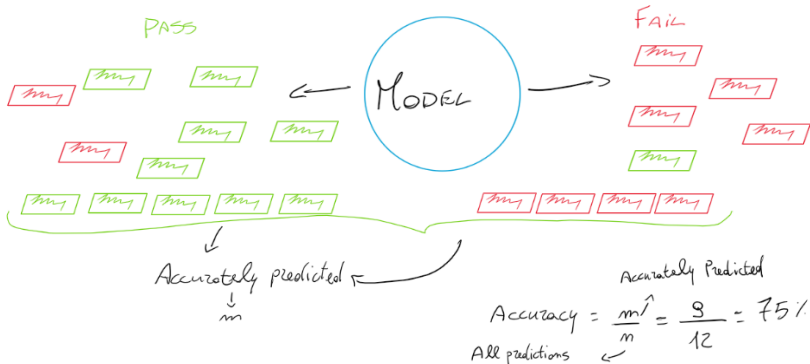


Figura 1.12 Nella figura è rappresentato un modello che predice due classi, fail e pass.

Da questi valori derivano altre metriche importanti.

La **precisione** misura tra tutte le predizioni positive, quante sono corrette.

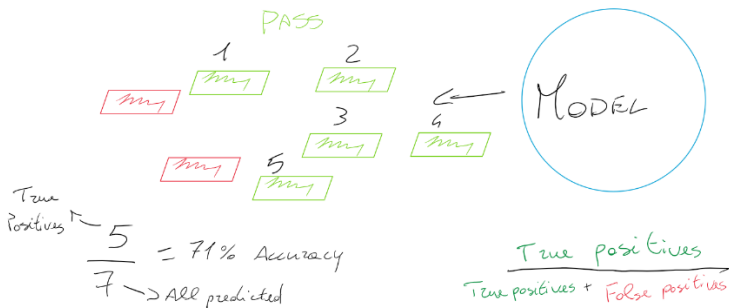


Figura 1.13 Nella figura è rappresentato uno schema per calcolare la precisione del modello.

Il **recall** misura tra tutti i casi realmente positivi, quanti il modello riesce a trovare.

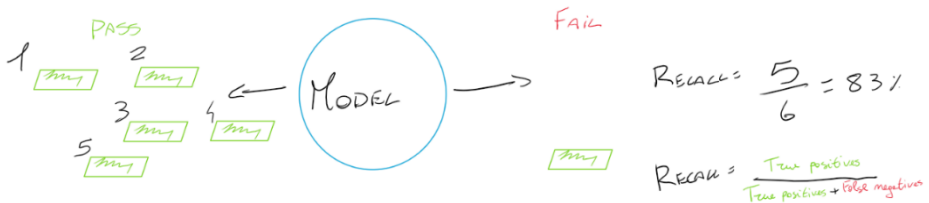


Figura 1.13 Nella figura è rappresentato uno schema per calcolare il recall del modello.

In molti problemi reali, come la diagnosi medica, non basta una sola metrica. Per questo si utilizza il **F1-score**, che combina precisione e recall in un unico valore, bilanciando entrambe con il calcolo:

$$\frac{2 * \text{precision} + \text{recall}}{\text{precision} + \text{recall}}$$

L'F1-score è comune usarlo quando abbiamo dataset sbilanciati.

Per i problemi di regressione invece abbiamo dei metodi diversi.

1.5.2 Valutazione dei modelli di regressione

Nei problemi di regressione il modello non predice categorie, ma valori numerici continui. Per valutarlo si misura quanto le sue previsioni si discostano dai valori reali.

Una delle metriche più usate è il **Mean Squared Error (MSE)**, che calcola la media degli errori elevati al quadrato tra valori predetti e valori reali. L'obiettivo è minimizzare questo valore.

Un'altra metrica è il **Root Mean Squared Error (RMSE)**, che è la radice quadrata del MSE. Ha il vantaggio di avere la stessa unità di misura del valore che stiamo predicendo, rendendo l'errore più interpretabile.

Infine c'è il **Mean Absolute Error (MAE)**, che calcola la media del valore assoluto degli errori. A differenza del MSE, non penalizza in modo eccessivo gli errori molto grandi.

1.6 Come impara una macchina

Abbiamo visto i tipi di apprendimento delle macchine in base ai dati disponibili, ma la macchina come impara?

Le macchine imparano modificando dei parametri al loro interno. Questi parametri descrivono il comportamento del modello. L'algoritmo di apprendimento avviene tramite diversi step:

1. Il modello fa una previsione che sarà ottenuta con parametri inizializzati casualmente, quindi sbagliata.
2. Calcola la **funzione di perdita**, ovvero un valore che determina **quanto** il modello sbaglia.
3. Aggiorna iterativamente i propri parametri per ridurre l'errore, utilizzando un algoritmo di ottimizzazione che li modifica in base alla funzione di perdita.
4. Ripete questi step finché l'errore si riduce. Il numero di volte che viene ripetuto l'addestramento è chiamato **epoca**.

Una volta completati questi step si va alla valutazione del modello sul dataset di test precedentemente diviso da quello di training (1.5).

1.6.1 Funzioni di perdita

La funzione di perdita viene calcolata tramite diversi metodi, in base al tipo di problema, se abbiamo una regressione si

usano spesso **Mean Square Error (MSE)**, **Mean Absolute Error (MAE)**, **Root Mean Square Error (RMSE)**, che misurano la differenza tra il valore predetto dal modello e il valore corretto.

$$\begin{aligned}MSE &= \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \\MAE &= \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \\RMSE &= \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}\end{aligned}$$

In cui y_i è il valore reale mentre $\hat{y}_i$ è il valore predetto dal modello.

Per problemi di classificazione binaria si usa la funzione **Binary Cross-Entropy Loss**, che misura la performance di un modello la cui uscita è un valore di probabilità compreso tra 0 e 1.

$$BCE = -\frac{1}{n} \sum_{i=1}^n [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)]$$

Dove y è l'etichetta corretta (0/1) e p è la probabilità predetta. Per calcolare l'errore in classificazione **multiclasse**, si usa la **Categorical Cross-Entropy Loss**, ovvero un'estensione di quella binaria.

$$CCE = - \sum_{i=1}^C y_i \log(\hat{y}_i)$$

Dove y_i è l'etichetta corretta della classe i , $\hat{y}_i$ è la probabilità e C il numero di classi.

Il passo successivo consiste nell'aggiornamento dei parametri del modello.

1.6.2 Aggiornamento dei parametri

Una volta calcolata la funzione di perdita, il modello utilizza questo valore per aggiornare i propri parametri in modo da ridurre l'errore nelle iterazioni successive. I modelli di machine learning hanno parametri diversi, ad esempio, un algoritmo di regressione lineare ha come parametri i **coefficienti** dell'equazione, mentre le reti neurali hanno **pesi** e **bias**.

Per aggiornare i parametri del modello, si calcola il gradiente della funzione di perdita rispetto ai parametri. Il gradiente indica **come cambiare ogni parametro per ridurre l'errore**. Guida il modello nella direzione in cui muoversi (aumentare o diminuire i valori) per migliorare le predizioni.

Capitolo 2 Modelli di Machine Learning Supervisionato

2.1 Regressione Lineare

La regressione lineare è l'algoritmo base per prevedere valori numerici continui. Trova la relazione tra la variabile target (y) e una o più variabili indipendenti (x).

Nel caso generale con più feature, il modello è descritto da:

$$y = f(x)$$

Dove x rappresenta l'insieme delle caratteristiche (colonne del dataset), mentre y è il valore da prevedere (target). Ad esempio, se vogliamo prevedere il prezzo di una casa in base a variabili come i metri quadri, possiamo usare questo tipo di algoritmo.

Area (x)	Valore (y)
70	156.000€
120	260.000€
30	110.000€
150	300.000€
80	??
160	??

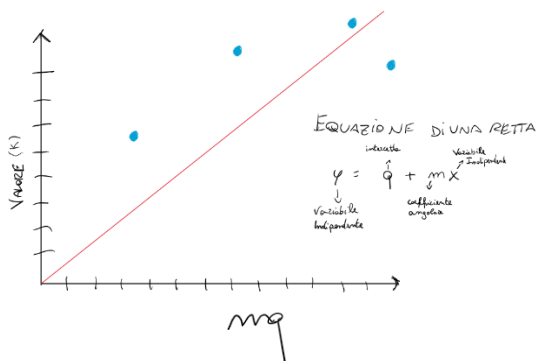


Figura 1.14 Nella figura è rappresentato l'algoritmo di regressione lineare con un esempio che ha come

caratteristica i metri quadri di una casa e come variabile indipendente il suo valore.

In un modello lineare se le caratteristiche sono più di una, questa relazione tra i dati è una somma pesata.

$$f(x) = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

L'intercetta (**bias**) stabilisce il valore di base della predizione, cioè il punto da cui parte il modello anche quando le feature sono nulle.

I coefficienti (**weights**) sono i parametri che la regressione deve ottimizzare per trovare la relazione migliore tra le caratteristiche e il target. Ogni coefficiente rappresenta l'impatto di una singola caratteristica sulla previsione finale. Aumentando il valore di un peso, aumenta anche l'influenza di quella variabile sul risultato. Nell'esempio della casa, un aumento dei metri quadri (x) porta generalmente a un aumento del valore (y).

Sostituendo i valori nella formula otteniamo l'equazione del modello:

$$y = wx + b$$

Dove **y** è il valore da predire, **w** è il peso associato alla feature e **b** è il bias.

Progetto 1 Regressione Lineare da zero con NumPy

Per questo progetto è richiesta una competenza con Python e la libreria NumPy, che permette di usare funzioni matematiche e vettori in Python.

Vogliamo creare un modello che in base agli anni di esperienza e allo stipendio dei miei dipendenti, quando si aggiunge un nuovo dipendente ipotetico, deve prevedere il suo stipendio.

```
import numpy as np

# Anni di esperienza (array di numeri)
X = np.array([24,14,5,15,26,35,8,0,1,16,17,37,4,6])
# Stipendio basato su anni di esperienza (inventato)
y =
np.array([2880,2000,1700,2230,2993,3012,1892,1100,1231,2103,2189,3231,1620,2018])
```

Iniziamo creando due array, X sarà la nostra variabile indipendente che rappresenta lo storico degli anni di esperienza dei miei dipendenti, mentre y sarà l'obiettivo della predizione quindi li stipendi corrispondenti ai dipendenti. Impostiamo l'equazione base inizializzando i parametri a 0

```
m = 0 # Coefficiente angolare (w)
q = 0 # Intercetta
y_pred = q + m * X
y_pred
OUTPUT:
array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0])
```

Con i parametri impostati a o la predizione sarà o.

Ora dobbiamo addestrare l'algoritmo di regressione con il ciclo di addestramento (1.6).

```
def derivatives(x, y, y_pred):  
    dq = np.mean(y_pred - y)  
    dm = np.mean((y_pred - y) * x)  
    return dq, dm
```

Usiamo questa funzione per derivate della funzione di perdita rispetto ai parametri che servono per l'aggiornamento dei parametri.

```
epochs = 100000  
lr = 0.0001  
n = len(X)  
for epoch in range(epochs):  
    y_pred = q + m * X  
    loss = np.mean((y_pred - y)**2)  
    dq, dm = derivatives(X, y, y_pred)  
    q = q - (lr * dq)  
    m = m - (lr * dm)  
    if epoch % 10 == 0:  
        print(f"Epoca: {epoch} \n Derivata Q: {dq} \n  
Derivata M: {dm} \n Nuovi parametri: Q: {q}, M: {m} \nLoss  
{loss}")
```

OUTPUT PRIMA RIGA: Epoca: 0

Derivata Q: -2157.0714285714284

Derivata M: -39119.357142857145

Nuovi parametri: Q: 0.21570714285714285, M:

3.9119357142857147

Loss 5060209.5

OUTPUT ULTIMA RIGA: Epoca: 99990

Derivata Q: -11.97848501375024


```
Derivata M: 0.5034662687751279
```

```
Nuovi parametri: Q: 1335.9674748533942, M: 54.46039836365676
```

```
Loss 32001.638050892798
```

Iniziamo definendo le epoche quindi quante volte il ciclo di addestramento deve essere ripetuto, poi iniziamo con il **Forward Pass**, ovvero la predizione del modello.

Calcoliamo la funzione di perdita che in questo caso è la **MSE** usando la funzione `.mean()` di NumPy che calcola la media, tra l'etichetta corretta e la predizione. Calcoliamo le derivate parziali di q ed m , chiamate **dq** e **dm** che verranno utilizzate per aggiornare i coefficienti q ed m . Per modificare i parametri bisogna sottrarre i coefficienti attuali, perché ci muoviamo nella direzione opposta al gradiente per minimizzare la perdita, alla moltiplicazione tra **learning rate** ovvero **la velocità con cui il modello impara** e la derivata del parametro. Dopo che abbiamo i pesi aggiornati possiamo ripetere il ciclo e quando raggiungeremo le 1000 iterazioni il modello avrà finito il ciclo di training.

Dopo l'allenamento vogliamo provare se il modello con dei nuovi dati sa generalizzare.

```
y_pred = []  
x_test = np.array([12,11,4,7,16,35,38,10,18,3,27,5,1,6])  
for x in x_test:  
    y_pred.append(q + m*x)  
y_pred
```

Creiamo un array nuovo di predizioni vuoto e un array con nuovi dati che rappresentano gli anni di esperienza di 15 dipendenti nuovi e per ogni dipendente facciamo la

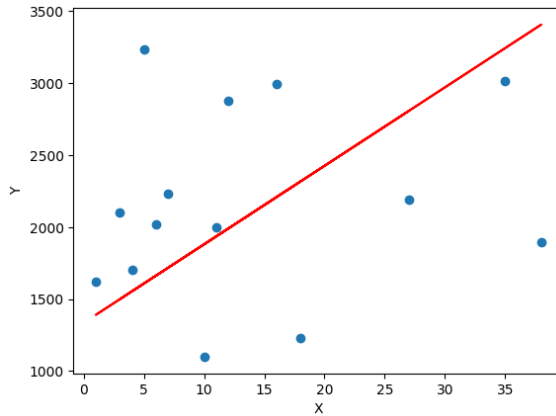
predizione usando l'equazione di base, con la differenza che ora q ed m saranno cambiati in modo da dare una predizione giusta, infatti otteniamo in output dei valori coerenti.

```
[np.float64(1989.4975974148892),  
 np.float64(1935.0376520858003),  
 np.float64(1553.818034782178),  
 np.float64(1717.1978707694448),  
 np.float64(2207.337378731245),  
 np.float64(3242.076339983934),  
 np.float64(3405.4561759712005),  
 np.float64(1880.5777067567114),  
 np.float64(2316.2572693894226),  
 np.float64(1499.3580894530892),  
 np.float64(2806.3967773512227),  
 np.float64(1608.277980111267),  
 np.float64(1390.4381987949114),  
 np.float64(1662.7379254403559)]
```

Per visualizzare correttamente se la retta di regressione prevede correttamente è meglio visualizzare i dati con matplotlib.

```
plt.scatter(x_test, y)  
plt.plot(x_test, m*x_test + q, color="red")  
plt.xlabel("X")  
plt.ylabel("Y")
```

In console ora visualizzeremo il grafico.



Come vediamo alcuni valori riesce a predirli correttamente mentre con altri ci è lontana. Questo essendo un esempio di implementazione usando solo matematica, il dataset era super ristretto quindi il modello non può imparare tanti pattern da avere un'accuratezza alta.

Progetto 2 Regressione Lineare con Sklearn e Pandas

In questo progetto useremo la libreria **scikit-learn** che è il master di tutti i progetti di machine learning perché facilita lo sviluppo dei modelli tramite l'utilizzo di metodi già preimpostati che evitano scritture a mano del codice. Oltre scikit-learn usiamo **pandas** che aiuta a manipolare i dati in formato .csv, Excel, json che normalmente non potremmo visualizzare in Python. Per i dataset il formato più utilizzato è il csv ovvero un tipo di file che presenta righe e colonne (1.3). Useremo il dataset di Kaggle (kaggle.com) che è una piattaforma di progetti riguardanti intelligenza artificiale e presenta una vasta gamma di dataset fatti da utenti che possiamo utilizzare nei nostri progetti, useremo oggi il dataset

(<https://www.kaggle.com/datasets/mohdshahnawazaadil/sales-prediction-dataset?resource=download>). Iniziamo il progetto importando pandas e visualizzando i nostri dati, si parte dalla visualizzazione dei dati perché serve comprenderli al meglio.

```
import pandas as pd
df = pd.read_csv('car_purchasing.csv', encoding='utf-8-sig')
df.info()
```

Creiamo il **DataFrame** contenente il nostro dataset in csv.

Definizione

Il DataFrame è un oggetto Python che crea pandas per lavorare e visualizzare i dati tabellari

Non lasciatevi confondere con ‘utf-8-sig’, di solito l’encoding si mette quanto il classico ‘utf-8’ non funziona e il dataset è di tipo diverso. Il metodo `info` restituisce le informazioni delle colonne del dataset

#	Column	Non-Null Count	Dtype
0	customer name	500 non-null	object
1	customer e-mail	500 non-null	object
2	country	500 non-null	object
3	gender	500 non-null	int64
4	age	500 non-null	float64
5	annual Salary	500 non-null	float64
6	credit card debt	500 non-null	float64
7	net worth	500 non-null	float64
8	car purchase amount	500 non-null	float64

Il computer lavora principalmente con numeri, quindi con **float (numeri con virgola)** o **int (numeri interi)**, i valori di tipo **object** sono spesso testo che richiede Pre-Processing con NLP. Il passo fondamentale per costruire un modello di machine learning è quello di capire quali dati ci servono in base a quello che vogliamo prevedere. In questo caso vogliamo sapere quanto spenderà in macchine il cliente (y), le informazioni fondamentali possono essere l’età, quanto guadagnano annualmente, il loro patrimonio netto, il debito che hanno sulla loro carta di credito. Rispetto al progetto precedente abbiamo più di una X quindi abbiamo uno spazio multidimensionale, in questo caso 3 dimensioni, quindi l’equazione della retta diventerà una somma pesata di più **x** e più **w**.

$$f(x) = w_1x_1 + w_2x_2 + w_3 + x_3 + b$$

A volte i dati grezzi non catturano bene la relazione. Creiamo nuove feature combinando quelle esistenti: questa pratica si chiama **feature engineering**. In questo caso possiamo ricavarci, ad esempio, il rapporto tra i debiti che il cliente ha e il suo introito annuale.

```
df['debt_to_income'] = df['credit card debt'] / df['annual Salary']
```

In questo modo ci siamo ricavati una caratteristica ulteriore che il modello userà per cogliere più pattern.

```
X = df[['age', 'annual Salary', 'credit card debt', 'net worth', 'debt_to_income']]
y = df['car purchase amount']
X.head()
```

Ora definiamo la X (caratteristiche) e la y (etichetta), visualizzando X.head() ovvero le prime 5 righe del dataset, avremo un resoconto dei dati che saranno passati al modello.

	age	Annual salary	Credit card debt	Net worth	Debt_to_income
0	41.851720	62812.09301	11609.380910	238961.2505	0.184827
1	40.870623	66646.89292	9572.957136	530973.9078	0.143637
2	43.152897	53798.55112	11160.355060	638467.1773	0.207447
3	58.271369	79370.03798	14426.164850	548599.0524	0.181758
4	57.313749	59729.15130	5358.712177	560304.0671	0.089717

Ora che abbiamo fatto feature engineering possiamo dividere il dataset in train e test usando il metodo di sklearn.

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2)
```

Abbiamo creato per entrambi gli array un set di train e un set di test dividendo per l'80% il train e 20% il test.

Ora siamo pronti ad istanziare il modello e allenarlo sul set di training.

```
from sklearn.linear_model import LinearRegression
lr = LinearRegression()
lr.fit(X_train, y_train)
```

Ora dopo aver allenato il modello possiamo testarlo.

```
lr.predict(X_test)
```

In output vedremo tutte le predizioni sul test di set che sarà un array lungo quanto il dataset di test. Questo output è lungo e non parla molto quindi possiamo fare quest'operazione per visualizzare un singolo cliente nel test di set.

```
nuovo_cliente = X_test.iloc[0:1]
predizione = lr.predict(nuovo_cliente)
print(f"Cliente: {nuovo_cliente}")
print(f"Predizione di spesa in auto: ${predizione[0][0]:.2f}")
```

Cliente: 263 Age: 54.491876 Annual Salary: 47684.46306
Credit card debit: 10128.76114 Net worth: 613372.8917
debt_to_income: 0.212412
Predizione di spesa in auto: \$48,303.26

Il vostro output sarà tabellare ma io l' ho messo in riga per farvi capire meglio.

Ora passiamo alla valutazione del modello (1.5) usando MSE e RMSE.


```
from sklearn.metrics import mean_squared_error, r2_score
import numpy as np

mse = mean_squared_error(y_test, y_pred)
print(f"MSE: {mse:,.2f}")

rmse = np.sqrt(mse)
print(f"RMSE: ${rmse:,.2f}")
OUTPUT:
MSE: 3.03
RMSE: $1.74
```

RMSE è più interpretabile di MSE, perché ci dice espressamente che il modello sta sbagliando in media di \$1.74. Guardando le metriche di valutazione capiamo che il modello è eccellente sbagliando in media solo di **~1.74\$**.

La regressione lineare abbiamo visto che prevede valori numerici continui tramite una somma pesata.

Ma cosa succede se vogliamo predire 2 casi??

Qui entra in gioco la classificazione: assegnare dati a categorie discrete (classi) di cui l'algoritmo più facile da comprendere è la **regressione logistica**.

2.2 Regressione Logistica

La regressione logistica è un algoritmo di classificazione, può prevedere l'appartenenza a una di 2 classi ed è chiamata **binaria** o l'appartenenza a più di 2 classi ed è chiamata **multinomiale**.

La regressione logistica per prevedere una probabilità compresa tra 0 e 1, utilizza la funzione **sigmoidea**.

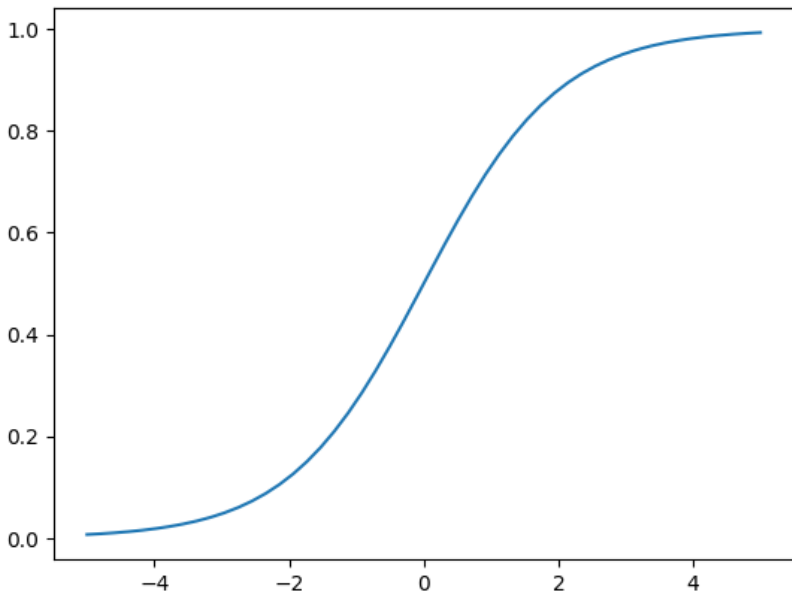


Figura 1.15 Grafico della funzione sigmoidea.

La funzione sigmoidea mappa qualsiasi numero reale in valori compresi tra 0 e 1 ed è descritta da questa formula.

$$P(z) = \frac{1}{1 + e^{-z}}$$

Dove z sarà il risultato dell'equazione della retta.

$$y = wx + b$$

Per più di due classi, sostituiamo la sigmoidea con **softmax**, che normalizza le probabilità di tutte le classi a somma 1. Dato un vettore di $z = [z_1, z_2, \dots, z_n]$ la funzione softmax è definita da:

$$\sigma(z_i) = \frac{e^{z_i}}{\sum_{j=1}^n e^{z_j}}$$

Dove e^{z_i} rappresenta l'esponenziale dei valori di input e $\sum_{j=1}^n e^{z_j}$ la somma di tutti i valori esponenziali (detta normalizzazione) e ogni output di $\sigma(z_i)$ rappresenta la probabilità predetta per la classe i .

Regressione Logistica From Scratch con NumPy

Ora creeremo un modello di classificazione binaria (**sigmoidea**) da zero, usando solo matematica con NumPy. Il modello sarà allenato su un array inventato che per X ore di studio, il modello deve prevedere se un alunno sarà promosso o bocciato.

```
import numpy as np
w = 0.1
b = -1
epochs = 5000
lr = 0.1
# Ore di studio (X)
X = np.array([1, 2, 2.5, 3, 3.5, 4, 4.5, 5, 5.5, 6, 6.5, 7,
7.5, 8, 8.5, 9, 9.2, 10, 10.5, 11])
# Risultato: 0 (Bocciato), 1 (Promosso)
y = np.array([0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1])
```

Iniziamo impostando pesi, bias, epoche e learning rate per poi creare i due array di dati.

```
def sigmoid(z):
    return 1/(1 + np.exp(-z))
def derivate(y_pred, y):
    dw = np.mean((y_pred - y) * X)
    db = np.mean(y_pred - y)
    return dw, db
def cross_entropy(y_pred, y):
    cost = -(y*np.log(y_pred) + (1-y) * np.log(1-
y_pred))
    return cost
```

Andiamo a definire la funzione sigmoidea replicando la formula. Calcoliamo le derivate dei parametri nello stesso modo di prima e calcoliamo anche la funzione di perdita

Cross-Entropy (1.6.1).

```
for epoch in range(epochs):
    z = w * X + b
    y_pred = sigmoid(z)
    loss = np.mean(cross_entropy(y_pred, y))
    dw, db = derivate(y_pred, y)
    w = w - dw * lr
    b = b - db * lr
    if epoch % 100 == 0:
        print(f"Epoca {epoch}: Loss {loss}")
```

Il ciclo di training rimane invariato da quello di regressione lineare tranne che applichiamo la funzione sigmoidea alla predizione dell'equazione (z).

```
x_eval = np.array([2, 5, 8])
z_test = w * x_eval + b
probabilita = sigmoid(z_test)
print(f"Probabilità di promozione per 2, 5, 8 ore: {probabilita}")
y_final_pred = sigmoid(w * X + b)
classi_predette = (y_final_pred >= 0.5).astype(int)
accuracy = np.mean(classi_predette == y) * 100
print(f"Accuracy del modello: {accuracy}%")
print(f"Predizioni: {classi_predette}")
print(f"Reali: {y}")
```

La valutazione la facciamo creando un vettore di 2,5,8 ore.

Calcoliamo le probabilità su `z_test` usando i nuovi parametri del modello. Calcoliamo le classi predette vere e calcoliamo l'accuracy. Riceveremo un output soddisfacente:

```
Probabilità di promozione per 2, 5, 8 ore: [0.00189156
0.23573247 0.98046883]
Accuracy del modello: 90.0%
Predizioni: [0 0 0 0 0 0 0 0 0 1 1 1 1 1 1 1 1 1]
Reali:      [0 0 0 0 0 0 0 0 1 0 1 1 1 1 1 1 1 1]
```

Possiamo ritenerci soddisfatti del nostro modello, osservando che sbaglia solo una predizione.

Regressione Logistica con Sklearn e Pandas

Svilupperemo un progetto che deve prevedere se un ragazzo è depresso in base alla sua presenza sui social media e altre caratteristiche.

```
import pandas as pd
df = pd.read_csv("data/Teen_Mental_Health_Dataset.csv")
df.info
```

Carichiamo il dataset e visualizziamo le info

```
Data columns (total 13 columns):
0   age                1200 non-null   int64
1   gender             1200 non-null   object
2   daily_social_media_hours  1200 non-null   float64
3   platform_usage     1200 non-null   object
4   sleep_hours        1200 non-null   float64
5   screen_time_before_sleep  1200 non-null   float64
6   academic_performance  1200 non-null   float64
7   physical_activity    1200 non-null   float64
8   social_interaction_level  1200 non-null   object
9   stress_level        1200 non-null   int64
10  anxiety_level       1200 non-null   int64
11  addiction_level     1200 non-null   int64
12  depression_label    1200 non-null   int64
dtypes: float64(5), int64(5), object(3)
```

Vediamo che abbiamo 3 colonne che contengono dati di tipo oggetto (solitamente dati testuali), i dati testuali non possono funzionare con algoritmi perché il computer non li capirebbe, per cui bisogna mappare i dati.

```
print(f"Values for platform_usage =  
{df['platform_usage'].unique()}")
```



```
print(f"Values for social_interaction_level =
{df['social_interaction_level'].unique()}")
OUTPUT:
Values for platform_usage = ['Instagram' 'TikTok' 'Both']
Values for social_interaction_level = ['low' 'high' 'medium']
```

Qui vediamo i diversi valori per queste colonne del DataFrame, questi devono essere modificati e resi numeri. Notiamo che possiamo mapparli in modo numerico come (0, 1 e 2).

```
df['gender'] = df['gender'].map({'male': 0, 'female': 1})
df['platform_usage'] =
df['platform_usage'].map({'Instagram': 0, 'TikTok': 1, 'Both': 2})
df['social_interaction_level'] =
df['social_interaction_level'].map({'low': 0, 'medium': 1, 'high': 2})
```

Mappiamo in 0,1 e 2 i diversi tipi con la funzione map di pandas.

```
from sklearn.model_selection import train_test_split

X =
df[['age', 'gender', 'daily_social_media_hours', 'platform_usage', 'sleep_hours', 'screen_time_before_sleep', 'academic_performance', 'physical_activity', 'social_interaction_level', 'stress_level', 'anxiety_level', 'addiction_level']]
y = df['depression_label']

X_train, X_test, y_train, y_test =
train_test_split(X, y, test_size=0.2)
```

Creiamo la matrice di caratteristiche e definiamo l'obiettivo della predizione(y), suddividendolo in train e test dataset con sklearn.

```
from sklearn.linear_model import LogisticRegression
lr = LogisticRegression()
lr.fit(X_train, y_train)
y_pred = lr.predict(X_test)
```

Creiamo il modello, alleniamolo sui dati di allenamento e creiamo la predizione su tutto il test.

```
from sklearn.metrics import accuracy_score,
recall_score, f1_score

acc = accuracy_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

print(f"Accuracy: {acc*100}%")
print(f"Recall: {recall}")
print(f"F1-Score: {f1}")
OUTPUT:
Accuracy: 98.33333333333333%
Recall: 0.6
F1-Score: 0.6
```

Abbiamo un accuracy altissima ma una recall ed un F1-score troppo bassi. Un recall basso sta a significare che il modello sta perdendo molti casi positivi reali mentre l'F1 score basso sta a significare uno scarso equilibrio tra recall e precisione. Questo accade con dataset sbilanciati.

Lavorare con dataset sbilanciati

Come visto nel modello di regressione logistica abbiamo un dataset sbilanciato quindi con più o che 1 o viceversa. Ci sono modi per risolvere questo problema nel Machine Learning e sono:

- **Over-sampling (Sovra campionamento):** Aumenta il numero di esempi nella classe di minoranza. Possiamo fare una duplicazione casuale dei dati o creazione di dati sintetici con **SMOTE** (Synthetic Minority Over-sampling Technique), che crea nuovi campioni basandosi sui vicini più prossimi.
- **Under-sampling (Sotto campionamento):** Riduce il numero di campioni della classe maggioritaria. Questa tecnica può causare la perdita di informazioni importanti.

Oppure possiamo assegnare pesi ad una classe per renderla più decisiva sulla previsione.

Ora aggiungiamo il parametro `class_weights` al modello precedente.

```
from sklearn.linear_model import LogisticRegression
lr = LogisticRegression(class_weight='balanced',
max_iter=1000)
lr.fit(X_train, y_train)
y_pred = lr.predict(X_test)
```

Abbiamo bilanciato le classi e alzato il numero di iterazioni di allenamento perché lo richiedeva sklearn.

```
Accuracy: 95.41666666666667%
Recall: 1.0
F1-Score: 0.47619047619047616
```

Ora notiamo che l'accuracy è leggermente diminuita e il recall si è alzato quindi il modello è migliorato perché trova tutti i positivi, ma notiamo ancora uno sbilanciamento tra precisione e recall. Nel machine learning migliorare una metrica spesso peggiora un'altra.